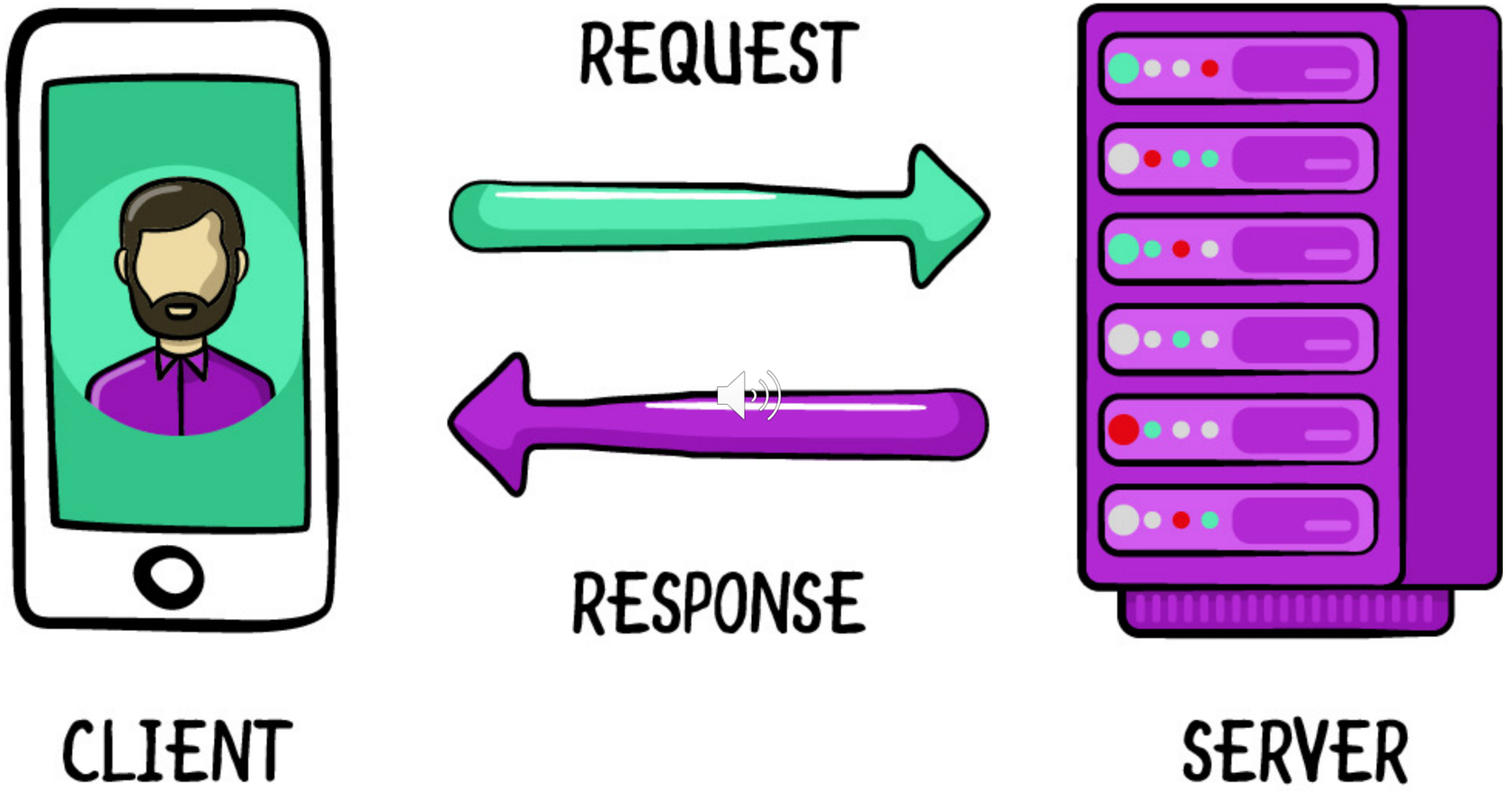
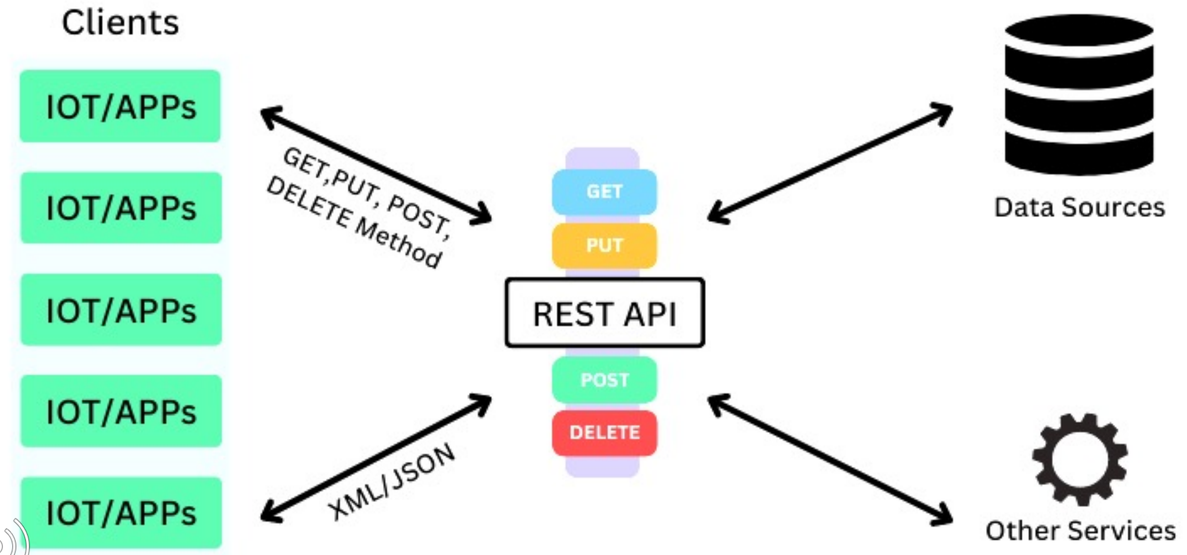


Сетевое взаимодействие

МИРЭА
МИРЭА
МИРЭА


REQUEST
RESPONSE
CLIENT
SERVER

Representational State Transfer Application Programming Interface - REST API



1. **Клиент-сервер**: Клиент и сервер функционируют независимо друг от друга, что позволяет их развивать и масштабировать отдельно.
2. **Статусный интерфейс**: Каждое взаимодействие между клиентом и сервером выполняется с помощью stateless запросов, что означает, что сервер не сохраняет состояние между запросами.
3. **Кэширование**: Ответы от сервера могут быть закэшированы, чтобы уменьшить количество запросов и повысить производительность.
4. **Многоуровневая система**: Архитектура может быть многоуровневой, что позволяет добавлять промежуточные слои (например, прокси-серверы и шлюзы) для управления запросами
5. **Единообразие интерфейса**: Для упрощения архитектуры все взаимодействия между клиентом и сервером осуществляются через стандартные методы HTTP (GET, POST, PUT, DELETE и т.д.).

Получение списка пользователей:
GET /api/v1/users

Удаление пользователя:
DELETE /api/users/1

Добавление нового пользователя:
POST /api/users
```
{
  "name": "John",
  "email": "john@example.com"
}
```

Обновление информации о пользователе:
PUT /api/users/1
```
{
  "name": "John Doe"
}
```

REST API. Недостатки

1.Избыточная и недостаточная выборка: Клиенты обычно получают все доступные поля ресурса, даже если им требуется только часть данных. Такая избыточная выборка может привести к увеличению времени отклика и пропускной способности канала. И наоборот, недостаточная выборка возникает, когда клиенту приходится делать несколько запросов к различным endpoints, чтобы получить необходимые данные.

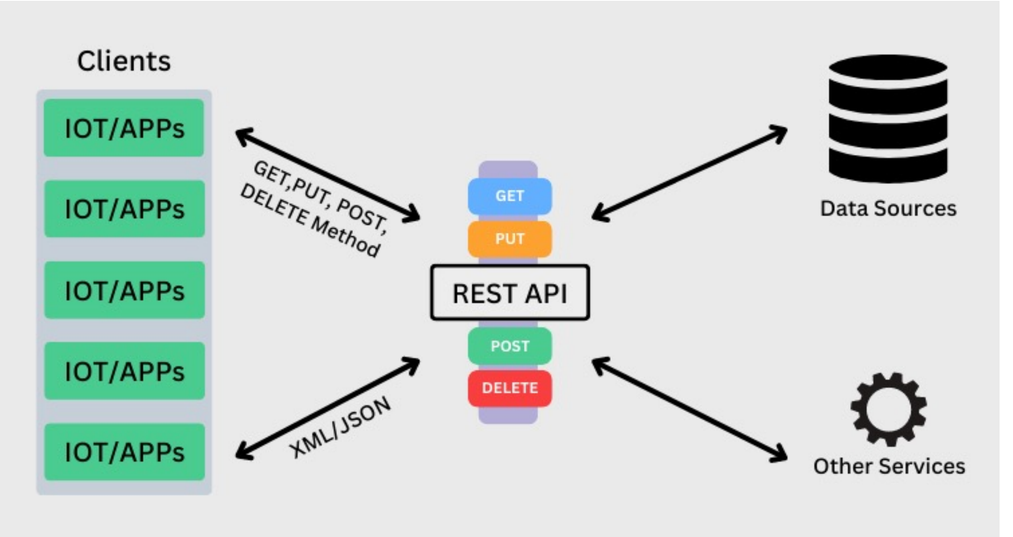
2.Меньшая гибкость: REST API обеспечивают ограниченную гибкость при запросах и манипуляциях с данными, поскольку используют структурированный, основанный на ресурсах подход с заранее определенными endpoints. Хотя такой подход более прост для понимания и реализации, он лишен гибкости, когда клиентам требуются более тонкие или детальные запросы.

REST API. Недостатки

3.Версионность: По мере роста и развития приложения управление изменениями в REST API может стать сложной задачей. Практика версионирования API различна; некоторые подходы могут привести к дублированию кода и головной боли при сопровождении.

4.Менее эффективен для сложных проектов: REST API может оказаться не лучшим выбором для приложений со сложными требованиями к данным и более сложными отношениями между ресурсами. При увеличении количества ресурсов и связей управление несколькими endpoints и вложенными данными может стать неуправляемым.

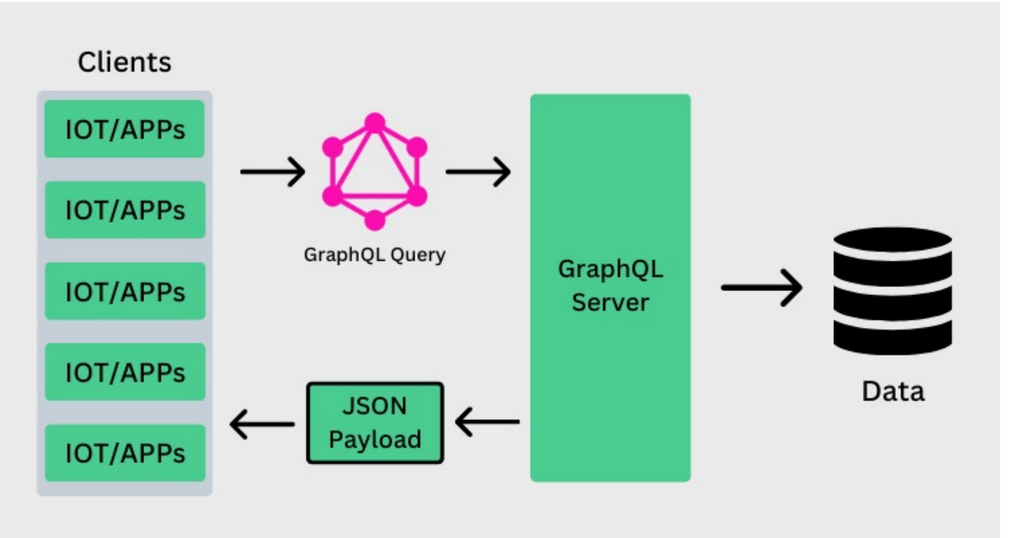
Representational State Transfer Application Programming Interface - REST API



библиотека для Android и Java от компании Square.

Помогает относительно легко получать и разбира

JSON (или другие структурированные данные).



Retrofit

библиотека для Android и Java от компании Square. Помогает относительно легко получать и разбирать JSON (или другие структурированные данные).

```
implementation 'com.squareup.retrofit2:retrofit:(insert latest version)'
implementation 'com.squareup.retrofit2:converter-gson::(insert latest version)'
```

```
addConverterFactory(GsonConverterFactory.create())
```

МИРЭА

Retrofit

Список готовых конвертеров:

Gson: com.squareup.retrofit2:converter-gson
Jackson: com.squareup.retrofit2:converter-jackson
Moshi: com.squareup.retrofit2:converter-moshi
Protobuf: com.squareup.retrofit2:converter-protobuf
Wire: com.squareup.retrofit2:converter-wire
Simple XML: com.squareup.retrofit2:converter-simplexml
Scalars (primitives, boxed, and String): com.squareup.retrofit2:converter-scalars

МИРЭА

Retrofit

Для работы с Retrofit понадобятся три класса.

- POJO (Plain Old Java Object) или Model Class - json-ответ от сервера нужно реализовать как модель;
- Retrofit - класс для обработки результатов. Требуется указать базовый адрес в методе baseUrl();
- Interface - интерфейс для управления адресом, используя команды GET, POST и т.д.

МИРЭА

Retrofit

jsonschema2pojo

Star 5,661 Tweet

Generate Plain Old Java Objects from JSON or JSON-Schema.

```
{
  "type":"object",
  "properties": {
    "foo": {
      "type": "string"
    },
    "bar": {
      "type": "integer"
    },
    "baz": {
      "type": "boolean"
    }
  }
}
```

Package: com.example

Class name: Example

Source type:
- ○ JSON Schema ◉ JSON
- ○ YAML Schema ○ YAML

Annotation style:
- ○ Jackson 2.x ◉ Gson
- ○ Moshi ○ None

- ☐ Generate builder methods
- ☐ Use primitive types
- ☐ Use long integers
- ☑ Use double numbers
- ☐ Use Joda dates
- ☑ Include getters and setters
- ☐ Include constructors
- ☐ Include `hashCode` and `equals`
- ☐ Include `toString`
- ☐ Include JSR-303 annotations
- ☑ Allow additional properties
- ☐ Make classes serializable
- ☐ Make classes parcelable
- ☐ Initialize collections

Property word delimiters: - _

Preview Zip

Preview

Copy to Clipboard

```
----------------------------------com.example.Bar.java----------------------------------

package com.example;

import javax.annotation.Generated;
import com.google.gson.annotations.Expose;
import com.google.gson.annotations.SerializedName;

@Generated("jsonschema2pojo")
public class Bar {

@SerializedName("type")
@Expose
private String type;

public String getType() {
return type;
}

public void setType(String type) {
this.type = type;
}

}
----------------------------------com.example.Baz.java----------------------------------

package com.example;

import javax.annotation.Generated;
import com.google.gson.annotations.Expose;
import com.google.gson.annotations.SerializedName;

@Generated("jsonschema2pojo")
public class Baz {

@SerializedName("type")
@Expose
private String type;

public String getType() {
return type;
}

public void setType(String type) {
this.type = type;
}

}
```

МИРЭА

Retrofit

```
import retrofit2.Call;

public interface APIService {
    @POST("list")
    Call<Repo> loadRepo();
    @GET("get_all_users")
    List<User> getAllUsers();

}
```

МИРЭА

Retrofit

Аннотация	Описание
@GET()	GET-запрос для базового адреса. Также можно указать параметры в скобках
@POST()	POST-запрос для базового адреса. Также можно указать параметры в скобках
@Path	Переменная для замещения конечной точки, например, **username** подставится в **{username}** в адресе конечной точки
@Query	Задаёт имя ключа запроса со значением параметра
@Body	Используется в POST-вызовах (из Java-объекта в JSON-строку)
@Header	Задаёт заголовок со значением параметра
@Headers	Задаёт все заголовки вместе
@Multipart	Используется при загрузке файлов или изображений
@FormUrlEncoded	Используется при использовании пары "имя/значение" в POST-запросах
@FieldMap	Используется при использовании пары "имя/значение" в POST-запросах
@Url	Для поддержки динамических адресов

Retrofit

```
@GET("/users/{username}")
Call getUser(
        @Path("username") String userName
);
```

МИРЭА

Retrofit

```
@Headers({"Cache-Control: max-age=640000", "User-Agent: My-App-Name"})
@GET("/users/{username}")
Call getUser(
        @Path("username") String userName
);
```

МИРЭА

Retrofit

```
@Multipart
@POST("some/endpoint")
Call<Response> uploadImage(@Part("description") String
description, @Part("image") RequestBody image)
```

МИРЭА

Retrofit

```
@FormUrlEncoded
@POST("/some/endpoint")
Call<SomeResponse> someEndpoint(@FieldMap Map<String, String> names);
```

МИРЭА

Retrofit

```
public static final String BASE_URL = "http://api.example.com/";

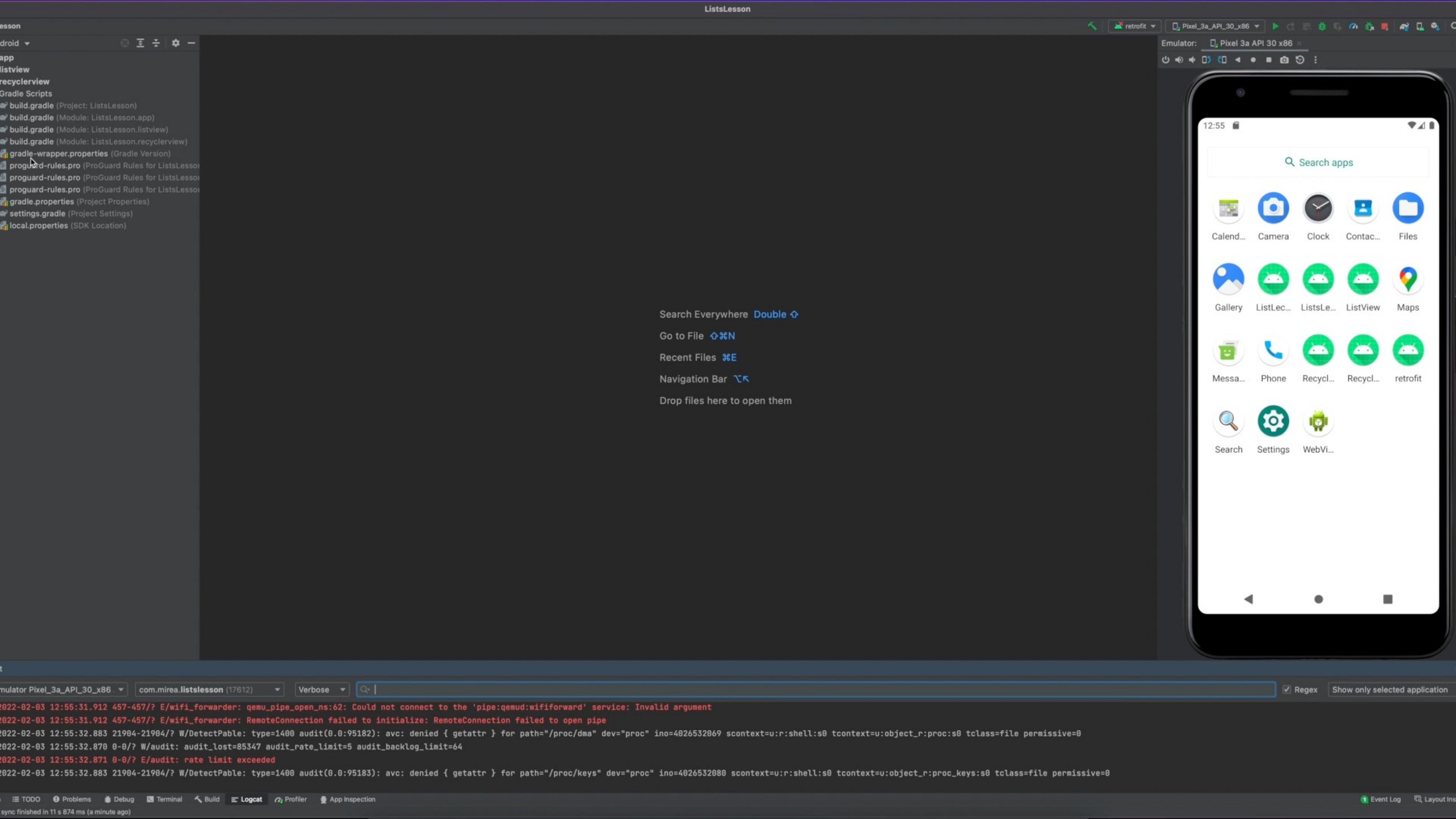
Retrofit retrofit = new Retrofit.Builder()
        .baseUrl(BASE_URL)
        .addConverterFactory(ProtoConverterFactory.create())
        .addConverterFactory(GsonConverterFactory.create())
        .build();

UserService userService = retrofit.create(UserService.class);

                                                    ....

if (response.isSuccessful()) {
    ... // код для успешного случая
    } else {
        switch(response.code()) {
            case 404:
                // страница не найдена. можно использовать ResponseBody
                break;
            case 500:
                // ошибка на сервере. можно использовать ResponseBody
                break;
        }
    }
```

МИРЭА


ListsLesson
app
listview
recyclerview
Gradle Scripts
build.gradle (Project: ListsLesson)
build.gradle (Module: ListsLesson.app)
build.gradle (Module: ListsLesson.listview)
build.gradle (Module: ListsLesson.recyclerview)
gradle-wrapper.properties (Gradle Version)
proguard-rules.pro (ProGuard Rules for ListsLesson
proguard-rules.pro (ProGuard Rules for ListsLesson
proguard-rules.pro (ProGuard Rules for ListsLesson
gradle.properties (Project Properties)
settings.gradle (Project Settings)
local.properties (SDK Location)
Search Everywhere Double ⇧
Go to File ⇧⌘N
Recent Files ⌘E
Navigation Bar ⌥↖
Drop files here to open them
Emulator: Pixel 3a API 30 x86
12:55
Search apps
Calend...
Camera
Clock
Contac...
Files
Gallery
ListLec...
ListsLe...
ListView
Maps
Messa...
Phone
Recycl...
Recycl...
retrofit
Search
Settings
WebVi...
com.mirea.listslesson (17612)
Verbose
Regex
Show only selected application
2022-02-03 12:55:31.912 457-457/? E/wifi_forwarder: qemu_pipe_open_ns:62: Could not connect to the 'pipe:qemud:wififorward' service: Invalid argument
2022-02-03 12:55:31.912 457-457/? E/wifi_forwarder: RemoteConnection failed to initialize: RemoteConnection failed to open pipe
2022-02-03 12:55:32.883 21904-21904/? W/DetectPable: type=1400 audit(0.0:95182): avc: denied { getattr } for path="/proc/dma" dev="proc" ino=4026532069 scontext=u:r:shell:s0 tcontext=u:object_r:proc:s0 tclass=file permissive=0
2022-02-03 12:55:32.870 0-0/? W/audit: audit_lost=85347 audit_rate_limit=5 audit_backlog_limit=64
2022-02-03 12:55:32.871 0-0/? E/audit: rate limit exceeded
2022-02-03 12:55:32.883 21904-21904/? W/DetectPable: type=1400 audit(0.0:95183): avc: denied { getattr } for path="/proc/keys" dev="proc" ino=4026532080 scontext=u:r:shell:s0 tcontext=u:object_r:proc_keys:s0 tclass=file permissive=0
TODO
Problems
Debug
Terminal
Build
Logcat
Profiler
App Inspection
Event Log
sync finished in 11 s 874 ms (a minute ago)